29

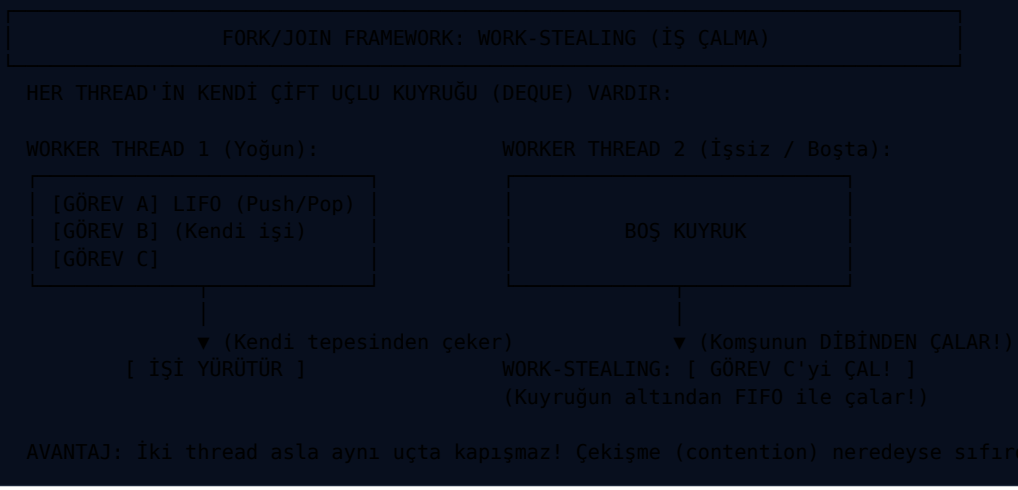
BÖLÜM: THREAD HAVUZLARI, EXECUTOR'LAR & WORK-STEALING | MODULE: THREAD POOLS, EXECUTORS & WORK-STEALING ALGORITHMS

Fork/Join Çerçevesi & Work-Stealing Algoritması: İş Çalma & Çift Uçlu Kuyruklar

Fork/Join Framework & The Work-Stealing Algorithm: Deque Poaching & Recursion

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

İki işçi düşünün: Ali ve Veli. İkisinin de önünde kendi özel iş tepsisi var. Ali'nin tepsisinde 100 tane evrak yığılı, Veli ise kendi işlerini hemen bitirip boşta kalmış. Veli oturup tembellik yapmaz; hemen Ali'nin tepsisine uzanır! Ama Ali tepsisinin tepesinden en taze evrakı alırken, Veli tepsisinin en altındaki en eski evrakı çalar (Work-Stealing). İkisi tepsisinin farklı uçlarından tuttukları için birbirlerinin eline çarpmazlar! Böylece hiçbir CPU çekirdeği boş kalmaz!

🇬🇧 ENGLISH (ELI5)

Imagine two postal sorters: Alice and Bob. Each owns a private Double-Ended Queue (Deque). Alice has 100 letters stacked; Bob clears his pile and sits idle. Under the Work-Stealing algorithm, Bob does not stay idle. He peeks at Alice's tray! Alice pushes and pops small subtasks from the TOP of her deque (LIFO, cache-friendly). Bob steals large root tasks from the BOTTOM of her deque (FIFO). Because they operate on opposite ends, thread contention is mathematically minimized!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ForkJoinPool ve Work-Stealing Derinliği: 1) Doug Lea'nın Work-Stealing Algoritması: Her worker thread kendi `WorkQueue` (çift uçlu kuyruk - Deque) yapısına sahiptir. Kendi görevlerini LIFO (Last-In-First-Out) sırasıyla işler; bu sayede CPU L1 önbellek uyumluluğu (Cache Locality) maksimize edilir. 2) Hırsızlık (Stealing): Boşta kalan thread başka bir rastgele worker'ın kuyruk dibinden FIFO ile çalar; en büyük iş parçacığı genellikle dipte yattığı için çalınan iş hırsızı uzun süre meşgul eder. 3) Java Paralel Akışlar (Parallel Streams): `list.parallelStream()` arka planda ortak `ForkJoinPool.commonPool()`'u kullanır! Eğer paralel stream içinde bloklayıcı bir I/O (ör. HTTP isteği veya DB sorgusu) yaparsanız, ortak havuzdaki tüm çekirdekleri kilitler ve tüm JVM'i çökertirsiniz!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

ForkJoinPool Mechanics & The Parallel Streams Hazard: 1) Dual-Ended Contention Isolation: Workers interact with their local array deque via LIFO push/pop, minimizing synchronization and maximizing CPU cache locality. Idle workers steal from victim deques via FIFO CAS operations on the opposite tail end. Large coarse-grained un-split tasks naturally reside at the tail, giving thieves substantial work chunks. 2) RecursiveTask vs RecursiveAction: Subclasses override `compute()`. `fork()` pushes asynchronously; `join()` executes cooperative work while waiting rather than blocking. 3) The `ForkJoinPool.commonPool()` Anti-Pattern: Java Parallel Streams execute on a shared global common pool sized to CPU cores. Executing blocking I/O inside parallel streams starves the common pool, deadlocking unrelated library components across the JVM.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: Fork/Join ile Büyük Dizi Toplamı (Böl ve Yönet / Divide and Conquer)
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveTask;

public class ParallelSumTask extends RecursiveTask<Long> {
    private static final int THRESHOLD = 10_000; // Eşik değer
    private final long[] array;
    private final int start, end;

    public ParallelSumTask(long[] array, int start, int end) {
        this.array = array;
        this.start = start;
        this.end = end;
    }

    @Override
    protected Long compute() {
        if (end - start <= THRESHOLD) {
            long sum = 0;
            for (int i = start; i < end; i++) sum += array[i];
            return sum;
        }

        int mid = start + (end - start) / 2;
        var leftTask = new ParallelSumTask(array, start, mid);
        var rightTask = new ParallelSumTask(array, mid, end);

        leftTask.fork(); // Sol parçayı başka bir thread'e çalması için bırak
        long rightResult = rightTask.compute(); // Sağ parçayı kendin hesapla
        long leftResult = leftTask.join();     // Sol parçayı bekle ve birleştir

        return leftResult + rightResult;
    }

    void main() {
        long[] data = new long[100_000];
        for (int i = 0; i < data.length; i++) data[i] = 1;

        try (var pool = new ForkJoinPool()) {
            long total = pool.invoke(new ParallelSumTask(data, 0, data.length));
            IO.println("Paralel Toplam Sonucu: " + total); // 100,000
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

MAANG Mülakat Kuralı: 'parallelStream() içine HTTP isteği veya veritabanı sorgusu yazabilir miyiz?' Yanıt: 'KESİNLİKLE HAYIR! parallelStream() tüm JVM'de paylaşılan ForkJoinPool.commonPool()'u kullanır. Birkaç yavaş I/O çağrısı havuzdaki tüm çekirdekleri tüketir ve sistemdeki diğer tüm paralel akışları felç eder!'

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Fatal Architecture Sin: 'Can we run HTTP calls or database queries inside parallelStream()?' Answer: 'NEVER! Parallel streams default to the shared ForkJoinPool.commonPool(). Blocking calls tie up common pool carrier threads, causing system-wide thread starvation across the entire JVM.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Fork/Join çerçevesi işlemci yoğun (CPU-bound) böl-ve-yönet algoritmaları içindir.

• Work-stealing, boşta kalan işçilerin diğer kuyrukların dibinden iş çalmasıdır.

• Bloklayıcı I/O işlemleri asla parallelStream() ve commonPool içine konulmamalıdır.

🎯 Key Takeaways for the Interview

• ForkJoinPool is optimized strictly for CPU-bound recursive divide-and-conquer workloads.

• Work-stealing allows idle workers to steal tasks from the tail of busy workers' deques.

• Never execute blocking I/O inside parallel streams or the shared common pool.